PROPOSAL TUGAS AKHIR

PENERAPAN MACHINE LEARNING, DENGAN PENDEKATAN *MULTICLASS CLASSIFICATION* UNTUK SISTEM REKOMENDASI FRAMEWORK WEB BERDASARKAN KARAKTERISTIK PROYEK

Dibuat Oleh:

412023024 – Geovano Yansen Jas

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI CERDAS

UNIVERSITAS KRISTEN KRIDA WACANA

JAKARTA 2026

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Pengembangan aplikasi berbasis web menyediakan banyak pilihan framework yang dapat digunakan oleh pengembang, seperti Django, Laravel, Flask, Spring Boot, dan framework lainnya. Banyaknya pilihan tersebut membuat proses menentukan framework yang sesuai tidak hanya bergantung pada ketersediaan teknologi, tetapi juga perlu mempertimbangkan karakteristik dan kebutuhan proyek yang akan dikembangkan.

Pengembangan aplikasi berbasis web menyediakan banyak pilihan framework yang dapat digunakan oleh pengembang, seperti Django, Laravel, Flask, Spring Boot, dan framework lainnya. Banyaknya pilihan tersebut membuat proses menentukan framework yang sesuai tidak hanya bergantung pada ketersediaan teknologi, tetapi juga perlu mempertimbangkan karakteristik fungsional maupun non-fungsional dari proyek yang akan dikembangkan.

Dalam praktik rekayasa perangkat lunak, pemilihan teknologi dan kerangka kerja (*framework*) merupakan salah satu keputusan arsitektural awal yang paling kompleks. Berdasarkan studi empiris oleh Farshidi et al. (2021) terhadap berbagai kasus industri, proses penentuan ekosistem teknologi sangat dipengaruhi oleh karakteristik proyek yang multidimensi, sehingga membutuhkan pertimbangan kriteria yang terstruktur untuk menghindari bias subjektif. Kompleksitas ini sejalan dengan temuan Schoenmakers et al. (2018) dalam penelitian terkait pemilihan artefak pengembangan, di mana pengembang terutama pengembang pemula sering kali mengalami kesulitan dalam menyeleksi komponen yang tepat akibat terbatasnya panduan komparatif yang objektif di antara puluhan alternatif teknologi yang tersedia. Ketidaksesuaian dalam memilih framework sejak awal proyek dapat memicu berbagai risiko kegagalan, mulai dari penurunan produktivitas tim, pembengkakan biaya pemeliharaan jangka panjang, hingga tingginya utang teknis (*technical debt*) yang menghambat skalabilitas sistem (Palomba et al.).

Permasalahan tersebut semakin relevan dengan dinamika industri perangkat lunak yang bergerak cepat. Data dari *Stack Overflow Developer Survey 2025* yang melibatkan lebih dari 49.000 responden dari 177 negara menunjukkan bahwa lanskap teknologi web terus mengalami pergeseran tren yang signifikan. Penggunaan Python dan *framework modern* seperti FastAPI mengalami peningkatan adopsi yang pesat, sementara integrasi alat bantu otomatisasi seperti Docker menjadi standar baru dalam infrastruktur

pengembangan. Kondisi ini memperlihatkan bahwa variasi ekosistem teknologi semakin luas, menuntut pengembang untuk dapat mencocokkan spesifikasi teknis proyek secara lebih presisi.

Di sisi lain, informasi mengenai pola penggunaan teknologi pada berbagai proyek perangkat lunak sebenarnya terekam secara luas dalam repositori *open-source*. Platform seperti GitHub menyimpan data historis yang kaya, mencakup bahasa pemrograman, struktur dependensi, basis data, hingga aktivitas kontributor. Pemanfaatan data empiris ini melalui pendekatan *Mining Software Repositories* (MSR) memungkinkan pola keputusan arsitektural masa lalu untuk dipelajari kembali secara objektif (Vidoni, 2022)

Berdasarkan permasalahan dan potensi tersebut, penelitian ini mengusulkan penerapan *machine learning* dengan pendekatan *multiclass classification* untuk membangun sistem rekomendasi framework web berdasarkan karakteristik proyek. Melalui sistem ini, data historis dari repositori perangkat lunak diolah untuk membantu mengotomatisasi proses pengambilan keputusan, sehingga pengguna dapat memperoleh rekomendasi framework secara terarah, objektif, dan berbasis data empiris.

1.2 Rumusan Masalah

Berdasarkan identifikasi masalah tersebut, rumusan masalah penelitian ini adalah bagaimana menerapkan machine learning dengan pendekatan multiclass classification untuk membangun sistem rekomendasi framework web berdasarkan karakteristik proyek?

Rumusan tersebut mencakup bagaimana memperoleh dataset proyek perangkat lunak dari repositori GitHub, bagaimana menentukan karakteristik yang dapat digunakan sebagai fitur, bagaimana membangun model multiclass classification, bagaimana mengevaluasi kemampuan model, serta bagaimana menerapkan model tersebut ke dalam sistem rekomendasi berbasis web.

1.3 Batasan Masalah

Penelitian ini berfokus pada rekomendasi framework untuk pengembangan aplikasi web. Framework-framework yang dijadikan sebagai target rekomendasi oleh *Machine Learning* diantaranya adalah Django, FastAPI, Ruby on Rails, Next.js, Flask, Spring Boot, Laravel, Symfony, dan Express.js. Kemudian Karakteristik proyek yang digunakan sebagai masukan sistem meliputi jenis aplikasi web, bahasa pemrograman, jenis basis data, kebutuhan penggunaan Docker, kebutuhan CI/CD, kebutuhan pengujian otomatis, penggunaan monorepo, jumlah pengembang, perkiraan jumlah pengguna, serta perkiraan lama pengembangan dan pemeliharaan aplikasi.

Sistem tidak bertujuan menentukan framework yang secara mutlak paling baik untuk seluruh proyek. Hasil yang diberikan merupakan rekomendasi berdasarkan pola yang dipelajari model dari data proyek yang digunakan dalam penelitian.

Penelitian berfokus pada *multiclass classification*, sehingga framework diperlakukan sebagai kelas keluaran. Model tidak digunakan untuk menghasilkan framework baru di luar kelas yang telah dipelajari selama proses pelatihan.

Dataset penelitian bersumber dari repositori perangkat lunak publik yang tersedia pada GitHub. Informasi yang digunakan berasal dari metadata repositori dan berkas konfigurasi atau dependensi yang dapat digunakan untuk mengidentifikasi framework dan karakteristik proyek.

1.4 Tujuan Penelitian

Penelitian ini bertujuan menerapkan machine learning dengan pendekatan multiclass classification untuk menghasilkan sistem yang mampu memberikan rekomendasi framework web berdasarkan karakteristik proyek.

Tujuan tersebut diwujudkan melalui pengumpulan dataset repository GitHub, ekstraksi karakteristik proyek, validasi framework, audit dan cleaning dataset, preprocessing, pembangunan model klasifikasi, evaluasi model, serta implementasi model terbaik ke dalam sistem berbasis web.

1.5 Manfaat Penelitian

Penelitian ini memberikan manfaat bagi mahasiswa, pengembang pemula, maupun pengembang perangkat lunak yang membutuhkan pertimbangan awal dalam menentukan framework untuk proyek aplikasi web. Sistem dapat membantu pengguna memperoleh rekomendasi berdasarkan karakteristik proyek yang dimasukkan. Pengguna tidak harus memahami proses machine learning karena proses pengolahan data dan prediksi dilakukan oleh sistem.

Bagi pengembang berpengalaman, sistem dapat digunakan sebagai alat bantu dalam membandingkan pilihan framework. Hasil rekomendasi tidak dimaksudkan sebagai keputusan mutlak, tetapi sebagai informasi tambahan dalam proses pengambilan keputusan.

Dari sisi akademis, penelitian ini memberikan penerapan machine learning pada permasalahan software engineering, khususnya pemanfaatan data *repository open-source* untuk membantu pengambilan keputusan teknologi. Penelitian mengenai sistem rekomendasi software engineering menunjukkan bahwa rekomendasi berbasis data dapat digunakan untuk membantu

pengembang menghadapi banyaknya pilihan artefak dan sumber daya perangkat lunak (Di Rocco et al., 2021)

BAB 2

LANDASAN TEORI

2.1 Software Engineering

Software engineering merupakan bidang yang berkaitan dengan proses pengembangan, pengelolaan, pemeliharaan, dan evaluasi perangkat lunak. Perkembangan teknologi perangkat lunak menghasilkan data dalam jumlah besar dari proses pengembangan yang dapat dimanfaatkan untuk penelitian berbasis data.

Kotti et al. (2023) menunjukkan bahwa machine learning telah banyak digunakan dalam berbagai aktivitas software engineering. Hal tersebut menunjukkan bahwa pendekatan berbasis machine learning dapat digunakan untuk membantu menyelesaikan permasalahan yang berkaitan dengan data dan pengambilan keputusan dalam software engineering.

2.2 Framework Web

Framework web merupakan kerangka kerja yang menyediakan struktur dan komponen untuk membantu pengembangan aplikasi berbasis web. Setiap *framework* memiliki karakteristik yang berbeda berdasarkan bahasa pemrograman, struktur aplikasi, dukungan database, pola pengembangan, dan kebutuhan deployment.

Dalam penelitian ini framework tidak hanya dipandang sebagai teknologi yang digunakan untuk membangun aplikasi, tetapi juga sebagai kelas target yang akan diprediksi oleh model machine learning.

2.3 Sistem Rekomendasi

Sistem rekomendasi merupakan sistem yang memberikan saran berdasarkan informasi tertentu. Dalam konteks software engineering, sistem rekomendasi dapat digunakan untuk membantu pengembang menemukan artefak, library, dokumentasi, atau sumber daya perangkat lunak yang sesuai dengan kebutuhan mereka. Di Rocco et al. (2021) menunjukkan penerapan sistem rekomendasi dalam software engineering melalui proyek CROSSMINER.

Penelitian ini menggunakan konsep tersebut pada permasalahan yang berbeda, yaitu memberikan rekomendasi framework pengembangan web berdasarkan karakteristik proyek.

2.4 Machine Learning

Machine learning merupakan pendekatan yang memungkinkan sistem mempelajari pola dari data untuk menghasilkan prediksi atau keputusan. Dalam software engineering, machine learning telah digunakan dalam berbagai permasalahan sehingga data pengembangan perangkat lunak dapat dimanfaatkan sebagai sumber pembelajaran model (Kotti et al., 2023).

Pada penelitian ini machine learning digunakan untuk mempelajari hubungan antara karakteristik proyek dan *framework* yang digunakan.

2.5 Supervised Learning

Penelitian ini menggunakan supervised learning karena dataset memiliki data masukan dan target yang telah diketahui. Karakteristik proyek digunakan sebagai variabel masukan, sedangkan *framework* digunakan sebagai target.

Model belajar berdasarkan pasangan karakteristik proyek dan *framework* yang terdapat dalam dataset sehingga dapat digunakan untuk melakukan prediksi terhadap proyek baru.

2.6 Classification

Classification merupakan proses pemetaan suatu data ke dalam kelas tertentu berdasarkan pola yang dipelajari dari data berlabel. Dalam penelitian ini, *framework* menjadi kelas yang akan diprediksi.

2.7 Multiclass Classification

Multiclass classification digunakan ketika target memiliki lebih dari dua kelas. Penelitian ini termasuk multiclass classification karena terdapat beberapa *framework* yang menjadi kelas target.

Dalam permasalahan multiclass, evaluasi model perlu mempertimbangkan performa masing-masing kelas. Martín De Diego et al. (2022) menjelaskan bahwa metrik klasifikasi memiliki karakteristik berbeda dan pemilihan metrik perlu disesuaikan dengan permasalahan, khususnya pada kasus multiclass dan data tidak seimbang.

2.8 Mining Software Repositories

Mining Software Repositories merupakan pendekatan yang memanfaatkan data dari repository perangkat lunak untuk memperoleh informasi dan bukti empiris mengenai perangkat lunak serta proses pengembangannya.

Vidoni (2022) menunjukkan bahwa penelitian MSR perlu memperhatikan proses pemilihan repository, proses ekstraksi data, kriteria penyaringan, dan potensi bias karena tahapan tersebut dapat memengaruhi hasil penelitian.

Pendekatan ini menjadi dasar dalam penelitian karena dataset diperoleh dari repository GitHub.

2.9 GitHub sebagai Sumber Data

GitHub merupakan platform yang menyediakan repository perangkat lunak dan berbagai informasi mengenai proyek. Data repository dapat mencakup kode, dependensi, struktur direktori, aktivitas commit, kontributor, dan metadata lainnya.

Pemanfaatan repository open-source sebagai sumber data penelitian telah menjadi bagian penting dalam Mining Software Repositories.

Namun, penelitian berbasis repository perlu memperhatikan karakteristik dan kualitas data yang digunakan. Gold dan Krinke (2022) menekankan bahwa penelitian yang melakukan mining terhadap software repository perlu memperhatikan aspek metodologis dan etika karena repository dapat mengandung data yang berkaitan dengan aktivitas pengembang.

2.10 Dataset

Dataset merupakan kumpulan data yang digunakan sebagai dasar proses pembelajaran machine learning. Pada penelitian ini dataset terdiri atas repository perangkat lunak yang telah melalui proses pengumpulan, ekstraksi, validasi, audit, dan cleaning.

Proses pembentukan dataset menjadi bagian penting karena kualitas data memengaruhi proses pembelajaran model. Vidoni (2022) menunjukkan pentingnya menjelaskan proses pemilihan dan ekstraksi data dalam penelitian berbasis MSR.

2.11 Feature dan Target

Feature merupakan variabel yang digunakan model sebagai dasar untuk menghasilkan prediksi, sedangkan target merupakan nilai yang ingin diprediksi.

Pada penelitian ini:

X = karakteristik proyek

Y = *framework*

Karakteristik seperti bahasa pemrograman, database, tipe aplikasi, Docker, CI/CD, pengujian, dokumentasi, monorepo, jumlah kontributor, dependensi, stars, forks, dan aktivitas repository digunakan sebagai kandidat feature.

2.12 Target Leakage

Target leakage terjadi ketika informasi yang berkaitan langsung dengan target masuk ke dalam data masukan model. Kondisi tersebut dapat menyebabkan model menghasilkan performa yang terlihat tinggi karena memperoleh informasi yang seharusnya tidak tersedia ketika melakukan prediksi.

Oleh karena itu, *framework_confidence* dan *framework_evidence* tidak digunakan sebagai feature. Kedua variabel tersebut hanya digunakan untuk memastikan framework yang terdeteksi dalam proses pembentukan dataset memiliki tingkat keyakinan yang memadai.

2.13 Preprocessing

Preprocessing dilakukan untuk mengubah data mentah menjadi data yang dapat digunakan oleh algoritma *machine learning*. Tahap *pre-processing* dapat mencakup pembersihan data, penanganan nilai kosong, transformasi variabel kategorikal, pengolahan variabel numerik, dan pemilihan fitur.

Pfob et al. (2022) menunjukkan bahwa *pre-processing*, *tuning*, dan perbandingan model merupakan bagian penting dalam pembangunan model machine learning yang berkualitas.

2.14 Class Imbalance

Class imbalance terjadi ketika jumlah data pada suatu kelas jauh lebih banyak dibandingkan kelas lainnya. Kondisi tersebut menjadi perhatian dalam penelitian ini karena distribusi *framework* tidak sepenuhnya sama.

Hasil dataset penelitian menunjukkan adanya perbedaan jumlah data yang cukup besar, terutama antara *framework* dengan jumlah *repository* tinggi dan Express.js yang memiliki jumlah data sangat sedikit.

Penelitian Carpita dan Golia (2023) menunjukkan bahwa *multiclass classification* dengan data tidak seimbang memiliki tantangan tersendiri, terutama dalam menangani kelas yang jarang muncul.

Oleh karena itu, penelitian perlu mempertimbangkan strategi penanganan imbalance serta menggunakan metrik yang mampu memperlihatkan performa pada masing-masing kelas.

2.15 Evaluasi Model

Evaluasi model dilakukan untuk mengetahui kemampuan model dalam melakukan prediksi terhadap data yang tidak digunakan selama proses pelatihan.

Accuracy dapat digunakan untuk melihat proporsi prediksi yang benar secara keseluruhan. *Precision* menunjukkan ketepatan prediksi pada suatu kelas, sedangkan recall menunjukkan kemampuan model menemukan data yang benar-benar termasuk dalam kelas tersebut. *F1-score* digunakan untuk melihat keseimbangan antara *precision* dan *recall*.

Pada *multiclass classification*, *macro-averaged metrics* penting diperhatikan karena memberikan bobot yang sama kepada setiap kelas. Hal tersebut menjadi relevan ketika dataset memiliki ketidakseimbangan kelas.

2.16 Confusion Matrix

Confusion matrix digunakan untuk melihat hubungan antara kelas sebenarnya dan kelas hasil prediksi. Matriks tersebut memungkinkan peneliti mengetahui framework mana yang berhasil dikenali dan *framework* mana yang sering tertukar.

Pada penelitian ini *confusion matrix* digunakan untuk mengetahui pola kesalahan klasifikasi antar-*framework*.

2.17 Hyperparameter Tuning

Hyperparameter tuning dilakukan untuk mencari konfigurasi parameter model yang dapat menghasilkan performa yang lebih baik. Liao et al. (2022) menunjukkan bahwa proses tuning dan optimasi dapat memengaruhi karakteristik performa model *machine learning*.

Jika penelitian menggunakan *Grid Search* atau metode *tuning* lainnya, proses tersebut perlu dilakukan hanya pada data training agar tidak menyebabkan kebocoran informasi dari data testing.

2.18 Explainable Recommendation

Sistem rekomendasi tidak hanya perlu menghasilkan prediksi, tetapi dalam beberapa kondisi juga perlu memberikan alasan yang dapat dipahami pengguna. Penelitian Naiseh et al. (2021) menunjukkan bahwa *explainable recommendation* berkaitan dengan kepercayaan pengguna terhadap hasil rekomendasi.

Dalam penelitian ini, aspek tersebut dapat diterapkan dengan menampilkan karakteristik proyek yang menjadi dasar rekomendasi atau tingkat probabilitas masing-masing *framework*. Dengan demikian, sistem tidak hanya memberikan hasil seperti “Django”, tetapi dapat memberikan informasi bahwa

berdasarkan karakteristik yang dimasukkan, Django memiliki probabilitas prediksi paling tinggi.

2.19 Penelitian Terdahulu

Penelitian terdahulu diperlukan untuk mengetahui posisi penelitian yang dilakukan dibandingkan dengan penelitian sebelumnya.

- Di Rocco et al. (2021) mengembangkan sistem rekomendasi untuk software engineering melalui proyek CROSSMINER. Sistem tersebut memanfaatkan data open-source untuk membantu pengembang memperoleh rekomendasi seperti library, dokumentasi API, dan fungsi API. Kelebihan penelitian tersebut adalah penerapan sistem rekomendasi secara nyata dalam software engineering. Namun, objek rekomendasi berfokus pada artefak dan sumber daya pengembangan, bukan pemilihan *framework* web berdasarkan karakteristik proyek.

- Kotti et al. (2023) melakukan tertiary study mengenai penerapan machine learning dalam software engineering dengan menganalisis 83 review dan 6.117 penelitian primer. Penelitian tersebut memperlihatkan luasnya penerapan machine learning dalam software engineering, tetapi bersifat kajian terhadap penelitian terdahulu dan tidak menghasilkan sistem rekomendasi *framework* web berdasarkan data repository.

- Vidoni (2022) melakukan systematic literature review terhadap penelitian Mining Software Repositories dan menunjukkan pentingnya proses pemilihan repository serta ekstraksi data secara sistematis. Penelitian tersebut memberikan dasar metodologis yang relevan untuk penelitian ini, tetapi tidak membangun model klasifikasi untuk rekomendasi *framework*.

- Carpita dan Golia (2023) membahas multiclass classification pada dataset yang tidak seimbang. Penelitian tersebut menunjukkan bahwa kelas dengan jumlah data yang sedikit dapat menimbulkan masalah dalam proses klasifikasi. Kondisi tersebut relevan dengan dataset penelitian ini karena jumlah repository pada setiap *framework* tidak sama. Namun, penelitian tersebut tidak membahas klasifikasi *framework* pengembangan web.

- Naiseh et al. (2021) membahas explainable recommendation dan hubungan antara desain penjelasan dengan tingkat kepercayaan pengguna. Penelitian tersebut menunjukkan pentingnya memberikan penjelasan terhadap hasil rekomendasi, tetapi tidak membahas pemilihan *framework* perangkat lunak.

Dengan demikian, penelitian terdahulu telah membahas sistem rekomendasi software engineering, machine learning untuk software engineering, Mining Software Repositories, multiclass classification, class imbalance, dan explainable recommendation. Akan tetapi, berdasarkan

penelitian yang ditinjau, masih terdapat ruang untuk menggabungkan aspek-aspek tersebut dalam suatu sistem yang secara khusus memberikan rekomendasi *framework* web berdasarkan karakteristik proyek.

2.20 Research Gap

Berdasarkan penelitian terdahulu, terdapat beberapa celah penelitian yang menjadi dasar penelitian ini.

1. penelitian sistem rekomendasi dalam software engineering telah digunakan untuk memberikan rekomendasi terhadap library, dokumentasi, API, dan artefak pengembangan lainnya. Namun, objek rekomendasi tersebut berbeda dengan penelitian ini yang menjadikan *framework* pengembangan web sebagai target rekomendasi.
2. penelitian machine learning dalam software engineering telah berkembang luas, tetapi kajian yang ada mencakup berbagai permasalahan software engineering dan tidak secara khusus berfokus pada klasifikasi *framework* web berdasarkan karakteristik proyek.
3. Penelitian Mining Software Repositories menunjukkan bahwa repository open-source dapat digunakan sebagai sumber data empiris, tetapi kualitas hasil penelitian sangat bergantung pada proses pemilihan repository dan ekstraksi data. Penelitian ini menerapkan proses tersebut secara khusus untuk membentuk dataset *framework* web.
4. permasalahan class imbalance telah banyak diteliti pada machine learning, tetapi pada penelitian ini ketidakseimbangan muncul pada kelas *framework* yang berasal dari repository perangkat lunak nyata. Oleh karena itu, distribusi kelas perlu dianalisis sebelum model dibangun.

Berdasarkan gap tersebut, penelitian ini menempatkan fokus pada hubungan antara karakteristik proyek → pola penggunaan framework → prediksi framework.

BAB 3

METODE PENELITIAN

3.1 Jenis Penelitian

Penelitian ini merupakan penelitian kuantitatif dengan pendekatan eksperimen. Penelitian dilakukan untuk menerapkan *machine learning* dengan pendekatan *multiclass classification* dalam membangun sistem rekomendasi *framework* pengembangan aplikasi web berdasarkan karakteristik proyek. Pendekatan kuantitatif digunakan karena penelitian menggunakan data proyek yang memiliki sejumlah karakteristik tertentu dan hasil pemodelan akan diukur menggunakan nilai evaluasi yang dapat dihitung secara objektif.

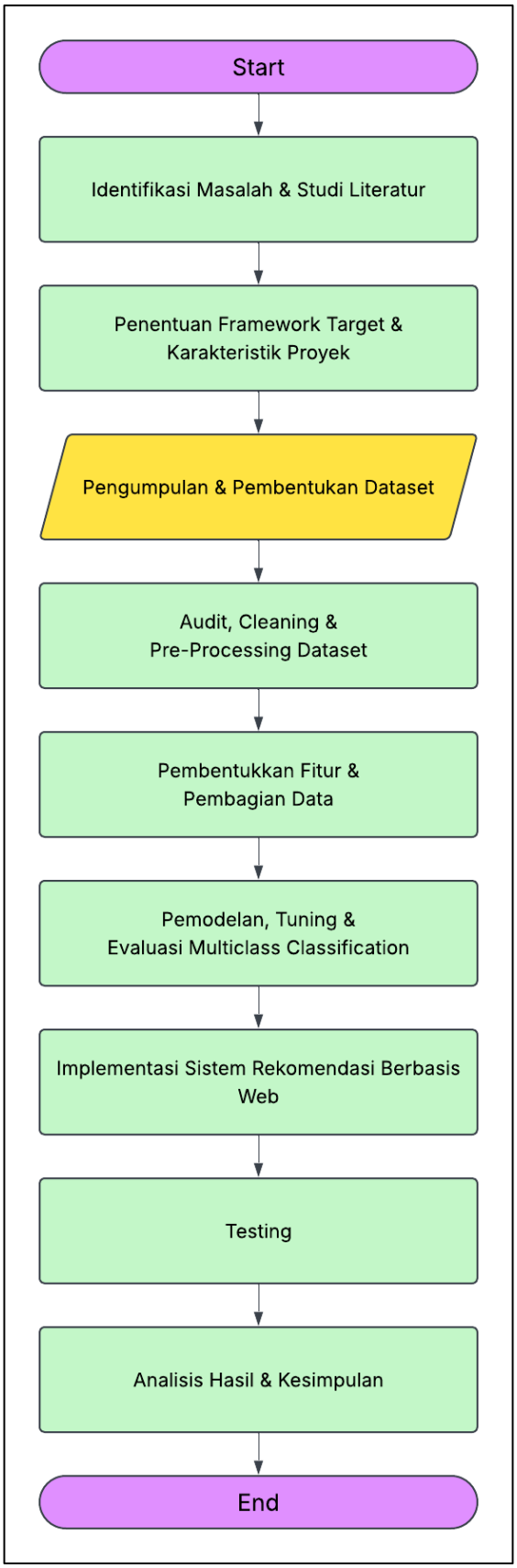
Penelitian ini berfokus pada hubungan antara karakteristik suatu proyek dengan *framework* yang digunakan pada proyek tersebut. Data proyek akan diperoleh dari repositori perangkat lunak *open source* yang tersedia secara publik. Karakteristik yang diperoleh dari repositori kemudian diolah menjadi data yang dapat digunakan oleh model *machine learning*. *Framework* yang digunakan pada proyek menjadi target atau kelas yang akan diprediksi oleh model.

Hasil akhir penelitian berupa sistem berbasis web yang memungkinkan pengguna memasukkan karakteristik proyek yang akan dikembangkan. Sistem kemudian mengolah masukan tersebut menjadi bentuk data yang sesuai dengan model dan menghasilkan rekomendasi *framework*. Rekomendasi tidak hanya berupa satu pilihan, tetapi dapat menampilkan *framework* utama dan beberapa alternatif berdasarkan tingkat kesesuaian hasil prediksi.

3.2 Tahap Penelitian

Penelitian ini dilaksanakan melalui beberapa tahapan yang saling berhubungan, mulai dari identifikasi masalah hingga analisis hasil dan penarikan kesimpulan. Tahapan penelitian disusun untuk memastikan proses pengembangan sistem rekomendasi *framework* dilakukan secara terarah, mulai dari penyusunan dasar penelitian, pembentukan dataset, pengembangan model machine learning, hingga penerapan model ke dalam sistem berbasis web.

Secara umum, tahapan penelitian yang dilakukan dapat dilihat pada Gambar 3.1



Gambar 3.1 Flowchart tahap penelitian

3.3 Sumber dan Karakteristik Data

Data penelitian bersumber dari repositori perangkat lunak *open source* yang tersedia secara publik pada GitHub. Pemilihan GitHub dilakukan karena repositori menyediakan informasi mengenai proyek perangkat lunak, bahasa pemrograman, *framework*, basis data, jumlah pengembang, aktivitas pengembangan, dependensi, serta karakteristik teknis lainnya yang dapat digunakan untuk menggambarkan suatu proyek.

Dataset penelitian direncanakan diperoleh melalui GitHub API sehingga proses pengumpulan dapat dilakukan secara terstruktur. Data yang dikumpulkan tidak hanya berupa nama repositori, tetapi juga informasi yang dapat menggambarkan karakteristik proyek dan *framework* yang digunakan.

Framework yang menjadi objek penelitian direncanakan mencakup beberapa *framework* pengembangan web yang memiliki penggunaan dan data repositori yang memadai. *Framework* yang digunakan sebagai kelas pada model akan ditentukan berdasarkan ketersediaan data yang memenuhi kriteria penelitian.

Data yang dikumpulkan akan digunakan untuk membentuk dua bagian utama, yaitu fitur atau karakteristik proyek sebagai masukan model dan *framework* sebagai target yang akan diprediksi.

Tabel 3.1 Sumber dan jenis data yang dikumpulkan

No	Jenis Data (Karakteristik Proyek)	Sumber Pengambilan Data Pada Github	Kategori Ekstraksi
1	Metadata Repositori (Stars,Forks, Ukuran Repositori, Bahasa Utama)	GitHub REST / GraphQL API	Diperoleh secara langsung melalui endpoint objek repositori.
2	Aktivitas Pengembangan (Jumlah Commit, Minggu Aktif, Kontributor)	GitHub API (Commits & Contributors Endpoint)	Diperoleh dengan menghitung riwayat commit dan daftar pengguna yang berkontribusi.
3	*Framework*, Basis Data, dan Jumlah Dependensi	Analisis Berkas Dependensi (Dependency Files)	Diekstraksi dengan membaca isi berkas seperti package.json, requirements.txt, atau pom.xml.
4	Penggunaan Containerization (Docker)	Analisis Struktur Direktori	Dideteksi dari keberadaan berkas Dockerfile atau docker-compose.yml pada repositori.
5	Penggunaan Continuous Integration / Deployment (CI/CD)	Analisis Struktur Direktori	Dideteksi dari keberadaan direktori konfigurasi seperti .github / workflows /, .travis.yml, atau .gitlab-ci.yml.
6	Keberadaan Pengujian (Testing)	Analisis Struktur Direktori dan Dependensi	Dideteksi dari keberadaan folder pengujian (contoh: /tests, /spec) atau library pengujian pada berkas dependensi.
7	Ketersediaan Dokumentasi	Analisis Struktur Direktori	Dideteksi dari keberadaan berkas README.md, dokumen Wiki, atau direktori /docs.
8	Struktur Repositori (Monorepo)	Analisis Struktur Direktori	Dideteksi dari keberadaan konfigurasi workspace (misalnya

			fitur workspaces pada package.json atau lerna.json).

3.4 Variabel Penelitian

Variabel penelitian dibagi menjadi variabel bebas (*input*) dan variabel terikat (*output framework*). Variabel masukan merupakan karakteristik proyek yang digunakan oleh model untuk menentukan *framework* yang paling sesuai. Sementara itu, variabel target merupakan *framework* yang digunakan sebagai hasil yang ingin diprediksi.

Karakteristik proyek tidak seluruhnya berasal dari satu sumber. Beberapa karakteristik dapat diperoleh secara langsung dari informasi repositori, sedangkan karakteristik lainnya diperoleh melalui analisis terhadap struktur dan berkas proyek.

Variabel yang digunakan dalam penelitian direncanakan meliputi bahasa pemrograman utama, basis data, tipe aplikasi, penggunaan Docker, penggunaan CI/CD, keberadaan pengujian, dokumentasi, penggunaan monorepo, jumlah kontributor, ukuran repositori, jumlah dependensi, serta tingkat aktivitas pengembangan.

Tabel 3.2 Devinisi Variabel Penelitian

Variabel	Jenis	Keterangan
primary_language	Bebas	Bahasa pemrograman utama proyek
database	Bebas	Basis data yang digunakan proyek
application_type	Bebas	Jenis aplikasi yang dikembangkan
has_docker	Bebas	Menunjukkan penggunaan Docker
has_ci_cd	Bebas	Menunjukkan penggunaan CI/CD
has_tests	Bebas	Menunjukkan keberadaan pengujian
has_documentation	Bebas	Menunjukkan keberadaan dokumentasi
monorepo	Bebas	Menunjukkan penggunaan struktur monorepo
contributors	Bebas	Jumlah kontributor proyek
repository_size_kb	Bebas	Ukuran repositori
dependency_count	Bebas	Jumlah dependensi yang terdeteksi
stars	Bebas	Jumlah stars repositori
forks	Bebas	Jumlah forks repositori
commits_last_52_weeks	Bebas	Jumlah commit dalam 52 minggu terakhir
active_weeks_last_52_weeks	Bebas	Jumlah minggu aktif dalam 52 minggu terakhir
framework	Terikat	*Framework* yang digunakan sebagai kelas prediksi

Variabel seperti *repository, repository_url, framework_confidence*, dan *framework_evidence* tidak digunakan sebagai fitur masukan model. Informasi tersebut hanya digunakan untuk keperluan identifikasi, pemeriksaan data, dan validasi hasil deteksi framework. Penggunaan informasi yang secara langsung menunjukkan framework dapat menyebabkan model memperoleh jawaban dari target yang seharusnya diprediksi.

3.5 Pengumpulan Dataset

Pengumpulan dataset dilakukan menggunakan GitHub API. Proses ini dilakukan secara otomatis menggunakan program yang dibuat dengan bahasa Python. Program akan melakukan pencarian repositori berdasarkan *framework* yang telah ditentukan dan kriteria pencarian yang telah ditetapkan.

Repositori yang dipilih merupakan repositori publik dan tidak diarsipkan. Selain itu, digunakan batas tertentu terhadap jumlah *stars* agar data yang dikumpulkan berasal dari proyek yang memiliki tingkat penggunaan atau perhatian komunitas yang cukup. Batas tersebut digunakan sebagai kriteria awal pengumpulan dan dapat disesuaikan berdasarkan kebutuhan penelitian.

Setiap repositori yang ditemukan akan diperiksa untuk memperoleh informasi mengenai bahasa pemrograman, *framework*, struktur proyek, dependensi, basis data, tipe aplikasi, serta karakteristik pengembangan lainnya. Informasi tersebut kemudian disimpan dalam dataset mentah.

Program pengumpulan juga menggunakan mekanisme pencatatan repositori yang telah diproses. Mekanisme tersebut bertujuan agar repositori yang sama tidak dikumpulkan kembali ketika proses pengumpulan dilanjutkan.

Dataset mentah yang dihasilkan pada tahap ini menjadi dasar untuk proses audit dan *cleaning*. Pada tahap proposal, jumlah akhir dataset belum ditetapkan karena jumlah data akan bergantung pada hasil proses pengumpulan dan penyaringan.

3.6 Audit dan Cleaning Dataset

Dataset yang telah dikumpulkan tidak langsung digunakan dalam proses *machine learning*. Dataset terlebih dahulu melalui tahap audit dan *cleaning* untuk memastikan bahwa data yang digunakan sesuai dengan tujuan penelitian.

Audit dilakukan untuk memeriksa struktur dataset, jumlah data, data duplikat, nilai kosong, ketidaksesuaian *framework*, serta keberadaan repositori yang tidak menggambarkan proyek aplikasi web secara tepat.

Repositori yang merupakan *framework* itu sendiri, tutorial, contoh pembelajaran, atau library dapat memberikan pola yang berbeda dari proyek aplikasi yang menjadi sasaran penelitian. Oleh karena itu, data tersebut akan ditandai dan dipertimbangkan untuk dikeluarkan dari dataset akhir.

Framework juga akan diperiksa berdasarkan informasi dependensi dan struktur proyek. Proses ini digunakan untuk memastikan bahwa label *framework* yang digunakan sebagai target memiliki dasar yang jelas.

Selain itu, pemeriksaan dilakukan terhadap tingkat keyakinan deteksi *framework*. Data dengan tingkat keyakinan yang tidak memenuhi batas yang telah ditentukan akan dipertimbangkan untuk dikeluarkan agar kualitas label target tetap terjaga.

Tabel 3.3 Kriteria audit dan cleaning dataset

Pemeriksaan	Keterangan
Duplikasi repositori	Menghapus data yang memiliki repositori sama
Framework tidak dikenali	Mengeluarkan data dari dataset
Framework / library / tutorial	Menandai dan menyaring data yang tidak sesuai
Label framework tidak meyakinkan	Mengeluarkan data berdasarkan batas confidence
Nilai kosong	Ditangani sesuai jenis variabel
Data tidak relevan	Dikeluarkan dari dataset

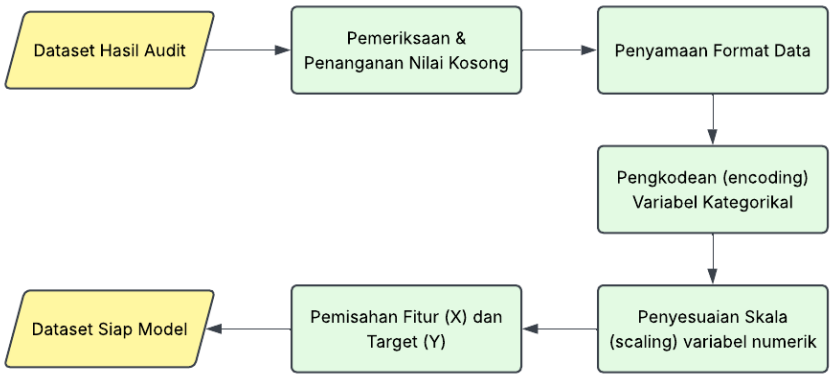
Hasil dari tahap ini adalah dataset yang telah disaring dan lebih sesuai untuk digunakan dalam penelitian. Dataset bersih tersebut kemudian digunakan pada tahap pre-processing.

3.7 Pre-Processing Dataset

Dataset yang telah melalui tahap audit selanjutnya dipersiapkan agar dapat digunakan oleh algoritma *machine learning*. Tahap ini dilakukan karena data yang berasal dari repositori memiliki bentuk yang berbeda-beda, sedangkan model membutuhkan data dengan struktur yang konsisten. *pre-processing* meliputi pemeriksaan kembali nilai kosong, penyamaan format data, pengubahan data kategorikal menjadi bentuk numerik, serta penyesuaian skala data apabila diperlukan.

Variabel kategorikal seperti bahasa pemrograman, basis data, dan tipe aplikasi tidak dapat langsung diproses oleh sebagian algoritma *machine learning*. Oleh karena itu, variabel tersebut akan diubah menjadi bentuk numerik menggunakan metode pengkodean yang sesuai.

Variabel numerik seperti jumlah kontributor, jumlah dependensi, ukuran repositori, jumlah *stars*, *forks*, dan aktivitas *commit* juga akan diperiksa. Apabila terdapat perbedaan rentang nilai yang terlalu besar, dilakukan normalisasi atau standardisasi sesuai kebutuhan algoritma yang digunakan. Pada tahap ini, target *framework* dipisahkan dari fitur agar model tidak menggunakan informasi target sebagai masukan.



Gambar 3.2 Alur preprocessing dataset

3.8 Pembentukkan Fitur

Pembentukan fitur dilakukan dengan menentukan karakteristik proyek yang dianggap relevan dalam proses pemilihan *framework*. Fitur yang digunakan tidak hanya mempertimbangkan bahasa pemrograman, tetapi juga kondisi dan kebutuhan teknis proyek.

Fitur masukan sistem dirancang untuk menggambarkan kondisi dan kebutuhan proyek web yang akan dikembangkan. Pertanyaan yang diberikan kepada pengguna dibuat dalam bentuk pilihan agar pengguna tidak perlu memahami istilah teknis secara mendalam. Setiap jawaban kemudian dikonversi menjadi nilai fitur yang sesuai dengan format data yang digunakan oleh model *machine learning.*

Dalam pemetaannya, berikut adalah list pertanyaan dan opsi jawaban yang akan digunakan sebagai kata kunci dalam penentuan *framework.*

a. Apa jenis aplikasi web yang ingin Anda bangun?
   - Web Application
   - API
   - CMS
   - Dashboard
   - E-commerce
   - CLI

b. Bahasa pemrograman apa yang ingin Anda gunakan untuk proyek ini?
   - Python

- PHP
- Java
- JavaScript
- TypeScript
- Ruby

c. Database apa yang akan digunakan dalam proyek?
   - PostgreSQL
   - MySQL
   - MongoDB
   - SQLite
   - Redis
   - Database relasional lainnya
   - Belum menentukan

d. Apakah proyek membutuhkan penggunaan Docker untuk menjalankan atau mendistribusikan aplikasi?
   - Ya
   - Tidak
   - Belum menentukan

e. Apakah proyek membutuhkan proses otomatis untuk melakukan pengujian dan deployment menggunakan CI/CD?
   - Ya
   - Tidak

f. Apakah proyek membutuhkan pengujian otomatis terhadap kode dan fitur aplikasi web?
   - Ya
   - Tidak

g. Apakah proyek akan mengelola beberapa aplikasi atau package dalam satu repository (monorepo)?
   - Ya

- Tidak

h. Berapa jumlah pengembang yang akan mengerjakan proyek ini?
   - 1 orang
   - 2–3 orang
   - 4–10 orang
   - Lebih dari 10 orang

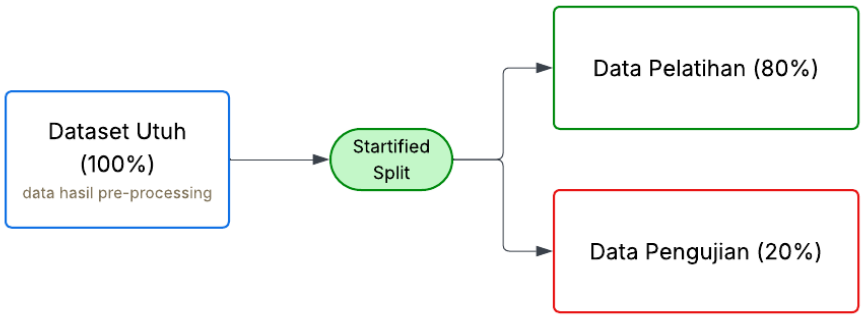
i. Berapa perkiraan jumlah pengguna aplikasi web?
   - Kurang dari 100 pengguna
   - 100–1.000 pengguna
   - 1.001–10.000 pengguna
   - 10.001–100.000 pengguna
   - Lebih dari 100.000 pengguna

j. Berapa lama aplikasi diperkirakan akan dikembangkan dan dipelihara?
   - Kurang dari 3 bulan
   - 3–6 bulan
   - 6–12 bulan
   - Lebih dari 1 tahun

3.9 Pembagian Data

Dataset yang telah diproses akan dibagi menjadi data pelatihan dan data pengujian. Data pelatihan digunakan untuk membangun pola hubungan antara karakteristik proyek dan *framework*. Sementara itu, data pengujian digunakan untuk mengetahui kemampuan model ketika diberikan data yang belum digunakan dalam proses pelatihan.

Pembagian data dilakukan secara terkontrol agar setiap kelas *framework* tetap memiliki keterwakilan yang memadai pada data pelatihan dan data pengujian. Pendekatan *stratified split* dapat digunakan agar proporsi setiap *framework* tetap relatif terjaga pada kedua bagian dataset.



Gambar 3.3 Ilustrasi Pembagian Dataset

Berdasarkan Gambar 3.3, Proporsi pembagian data akan ditentukan berdasarkan jumlah dataset akhir dan kebutuhan eksperimen. Salah satu skenario yang dapat digunakan adalah 80% data untuk pelatihan dan 20% untuk pengujian. Data pengujian tidak digunakan dalam proses pelatihan maupun penentuan parameter model. Hal tersebut dilakukan agar hasil evaluasi dapat menggambarkan kemampuan model terhadap data yang belum pernah dilihat sebelumnya.

3.10 Pemodelan Multiclass Classification

Pada penelitian ini, *machine learning* diterapkan menggunakan pendekatan *multiclass classification* dengan algoritma Random Forest. Pendekatan ini digunakan karena *framework* yang menjadi target rekomendasi terdiri atas lebih dari dua kelas, yaitu Django, FastAPI, Ruby on Rails, Next.js, Flask, Spring Boot, Laravel, Symfony, dan Express.js. Setiap data proyek dalam dataset memiliki satu *framework* yang digunakan sebagai label atau target klasifikasi.

Random Forest digunakan untuk mempelajari hubungan antara karakteristik proyek dengan *framework* yang digunakan pada proyek-proyek sebelumnya. Model dibangun melalui sejumlah *decision tree* yang dilatih menggunakan data pelatihan dengan proses pengambilan sampel dan pemilihan fitur secara acak. Hasil prediksi dari sejumlah *decision tree* tersebut kemudian digabungkan untuk menghasilkan keputusan klasifikasi (Breiman, L. 2001).

Fitur yang digunakan dalam proses pemodelan merupakan karakteristik proyek yang telah melalui tahap *pre-processing* dan pembentukan fitur. Karakteristik tersebut digunakan sebagai masukan untuk memprediksi kelas *framework*. Sementara itu, kolom yang secara langsung menunjukkan informasi *framework*, seperti *framework_confidence* dan

framework_evidence, tidak digunakan sebagai fitur karena dapat menyebabkan *target leakage*.

Setelah proses pelatihan selesai, model Random Forest menghasilkan prediksi *framework* untuk data proyek baru. Selain kelas dengan prediksi tertinggi, model juga dapat menghasilkan nilai probabilitas untuk setiap kelas *framework*. Nilai tersebut digunakan untuk mengetahui urutan hasil prediksi, sehingga *framework* dengan probabilitas tertinggi dapat dijadikan rekomendasi utama dan *framework* dengan probabilitas lebih rendah dapat dipertimbangkan sebagai alternatif.

Model yang telah dilatih selanjutnya dievaluasi menggunakan data pengujian. Hasil evaluasi digunakan untuk mengetahui kemampuan model dalam mengklasifikasikan *framework* berdasarkan karakteristik proyek. Model Random Forest yang memenuhi hasil evaluasi terbaik kemudian disimpan dan diintegrasikan ke dalam sistem rekomendasi berbasis web.

3.11 Mekanisme Rekomendasi

Mekanisme rekomendasi dimulai ketika pengguna memasukkan karakteristik proyek melalui sistem berbasis web. Karakteristik yang dimasukkan meliputi jenis aplikasi, bahasa pemrograman, database, kebutuhan Docker, kebutuhan CI/CD, kebutuhan pengujian otomatis, penggunaan monorepo, jumlah pengembang, perkiraan jumlah pengguna, serta estimasi lama pengembangan dan pemeliharaan aplikasi.

Data masukan pengguna selanjutnya disesuaikan dengan format fitur yang digunakan pada saat proses pelatihan model. Proses ini diperlukan agar struktur data yang diterima oleh model pada saat sistem digunakan memiliki bentuk yang sama dengan data pelatihan.

Selanjutnya, data tersebut diproses oleh model Random Forest yang telah dilatih. Model menghasilkan probabilitas prediksi untuk setiap *framework* target. *Framework* dengan probabilitas tertinggi ditempatkan sebagai rekomendasi utama, sedangkan beberapa *framework* dengan probabilitas berikutnya dapat ditampilkan sebagai alternatif.

Sebagai ilustrasi, apabila suatu proyek memiliki karakteristik tertentu dan model menghasilkan probabilitas prediksi Django sebesar 0,85, FastAPI sebesar 0,09, Flask sebesar 0,03, dan *framework* lainnya memiliki probabilitas yang lebih rendah, maka sistem menempatkan Django sebagai rekomendasi utama. FastAPI dan Flask dapat ditampilkan sebagai alternatif berdasarkan urutan probabilitas prediksi.

Nilai probabilitas tersebut menunjukkan tingkat keyakinan model dalam mengklasifikasikan data masukan ke dalam masing-masing kelas berdasarkan pola yang dipelajari dari dataset. Oleh karena itu, nilai probabilitas tidak diartikan sebagai ukuran mutlak bahwa suatu *framework* memiliki tingkat kecocokan tertentu secara objektif, melainkan sebagai hasil prediksi model terhadap pola data yang tersedia.

Agar hasil rekomendasi tidak hanya berupa nama *framework*, sistem juga menyusun penjelasan berdasarkan karakteristik proyek yang dimasukkan oleh pengguna. Penjelasan tersebut dibentuk menggunakan *knowledge base* yang memuat informasi mengenai karakteristik umum *framework* target. Informasi tersebut digunakan untuk menjelaskan alasan yang relevan terhadap rekomendasi yang diberikan.

Sebagai contoh, apabila pengguna memilih Python sebagai bahasa pemrograman dan hasil prediksi utama adalah Django, sistem dapat menjelaskan bahwa Django direkomendasikan karena sesuai dengan bahasa pemrograman yang dipilih. Karakteristik lain yang dimasukkan pengguna, seperti jenis aplikasi dan kebutuhan pengembangan, juga dapat digunakan sebagai bagian dari penjelasan rekomendasi.

Dengan mekanisme tersebut, hasil akhir sistem tidak hanya berupa satu nama *framework*. Sistem menghasilkan rekomendasi utama, alternatif *framework* berdasarkan urutan probabilitas prediksi, serta penjelasan yang membantu pengguna memahami alasan dari hasil rekomendasi tersebut.

3.12 Evaluasi Model

Setelah proses pelatihan menggunakan algoritma Random Forest, model dievaluasi untuk mengetahui kemampuan model dalam mengklasifikasikan *framework* berdasarkan karakteristik proyek. Evaluasi dilakukan menggunakan data pengujian yang tidak digunakan pada proses pelatihan. Karena penelitian menggunakan pendekatan multiclass classification, evaluasi dilakukan dengan melihat performa model pada seluruh kelas *framework*.

Evaluasi model menggunakan beberapa metrik, yaitu *accuracy, precision, recall*, dan *F1-score*. Accuracy digunakan untuk mengetahui proporsi prediksi yang benar terhadap seluruh data pengujian. Precision digunakan untuk mengetahui ketepatan model dalam memberikan prediksi pada suatu kelas, sedangkan recall digunakan untuk mengetahui kemampuan model dalam mengenali data yang sebenarnya termasuk dalam suatu kelas. *F1-score* digunakan sebagai ukuran yang menggabungkan precision dan *recall*. Metrik-metrik tersebut dapat digunakan pada

permasalahan multiclass classification dengan perhitungan terhadap masing-masing kelas dan metode rata-rata tertentu.

Selain metrik tersebut, digunakan *Macro F1-score* untuk melihat performa model dengan memberikan bobot yang sama terhadap setiap kelas framework. Penggunaan *Macro F1-score* penting karena jumlah data pada setiap kelas framework tidak sepenuhnya seimbang. Dengan demikian, performa pada kelas dengan jumlah data lebih sedikit tetap diperhitungkan dalam penilaian model.

Evaluasi juga dilakukan menggunakan confusion matrix untuk melihat pola kesalahan klasifikasi pada masing-masing *framework*. Confusion matrix menunjukkan jumlah data yang berhasil maupun tidak berhasil diklasifikasikan ke dalam setiap kelas, sehingga dapat digunakan untuk mengetahui framework yang paling sering tertukar oleh model.

Hasil evaluasi digunakan untuk menentukan apakah *model Random Forest* memiliki kemampuan klasifikasi yang memadai untuk digunakan dalam sistem rekomendasi framework web. Model yang telah dievaluasi kemudian digunakan pada tahap mekanisme rekomendasi untuk menghasilkan prediksi *framework* berdasarkan karakteristik proyek yang dimasukkan oleh pengguna.

3.13 Pengembangan Sistem Berbasis Web

Setelah model *machine learning* berhasil dibangun dan dievaluasi, tahap berikutnya adalah mengimplementasikan model tersebut ke dalam sistem berbasis web. Sistem ini berfungsi sebagai antarmuka yang memungkinkan pengguna memasukkan karakteristik proyek tanpa perlu memahami proses *machine learning* yang berlangsung di dalam sistem. Berdasarkan karakteristik yang diberikan, sistem akan melakukan pengolahan data, menjalankan model klasifikasi yang telah dilatih, kemudian menghasilkan rekomendasi framework yang sesuai.

Pengembangan sistem dilakukan dengan memisahkan bagian antarmuka pengguna dengan bagian pemrosesan model. Antarmuka digunakan untuk menerima informasi proyek dari pengguna, sedangkan bagian pemrosesan bertugas mengubah masukan tersebut menjadi format yang sesuai dengan model dan menghasilkan prediksi. Dengan pendekatan tersebut, pengguna tidak berinteraksi secara langsung dengan kode atau model *machine learning*, tetapi cukup memberikan informasi mengenai proyek yang ingin dikembangkan.

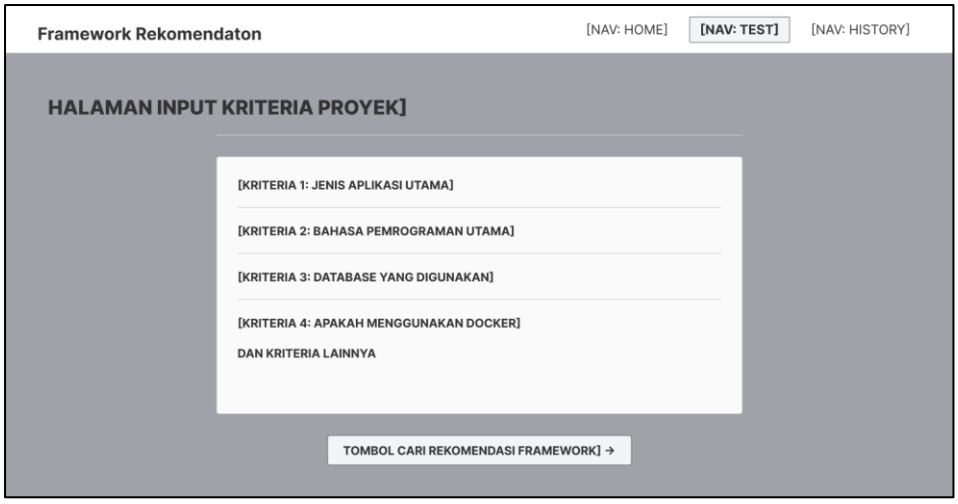
Sistem rekomendasi dikembangkan menggunakan Python sebagai bahasa pemrograman utama pada sisi pemrosesan, karena Python

digunakan dalam proses pengolahan dataset, pelatihan model, evaluasi model, serta implementasi model *machine learning*. Penggunaan bahasa yang sama antara tahap pelatihan dan implementasi model bertujuan untuk menjaga konsistensi proses pengolahan data.

Pada sisi *machine learning*, penelitian menggunakan pustaka Python yang sesuai dengan metode klasifikasi yang dipilih. Pustaka tersebut digunakan untuk melakukan *pre-processing*, pembentukan fitur, pelatihan model, penyimpanan model, dan proses prediksi ketika sistem digunakan.

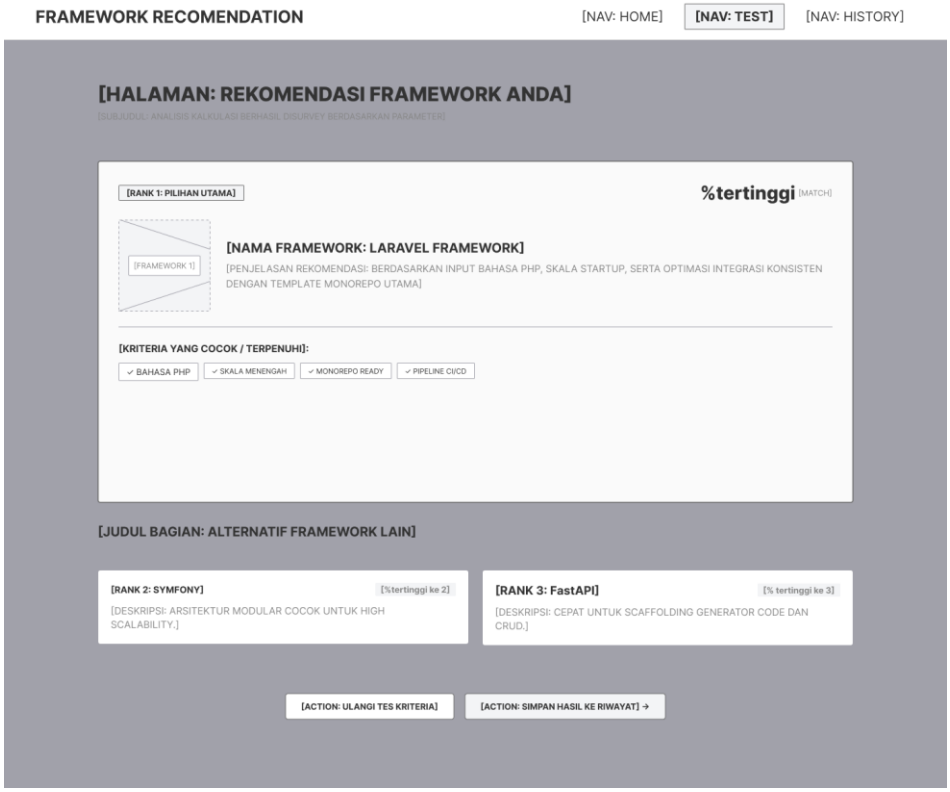
Untuk bagian antarmuka, sistem web dapat dikembangkan menggunakan HTML, CSS, dan JavaScript. HTML digunakan untuk membentuk struktur halaman, CSS digunakan untuk mengatur tampilan, sedangkan JavaScript digunakan untuk memberikan interaksi pada halaman ketika diperlukan. Antarmuka tersebut menjadi bagian yang berhubungan langsung dengan pengguna ketika memasukkan karakteristik proyek.

Untuk menghubungkan antarmuka web dengan model *machine learning*, digunakan pendekatan web application berbasis Python. Framework web yang digunakan adalah Flask, karena Flask memiliki struktur yang sederhana dan dapat digunakan untuk menyediakan *endpoint* yang menerima data proyek, menjalankan model, serta mengembalikan hasil prediksi kepada antarmuka web (A. C. Darmawan and L. Iswari. 2022).



Pada halaman input, pengguna akan diberikan pertanyaan mengenai karakteristik proyek. Setiap jawaban akan diproses menjadi fitur sesuai dengan format yang digunakan oleh model.

Setelah pengguna mengirimkan data, sistem melakukan *pre-processing* terhadap masukan tersebut dan meneruskannya kepada model. Model kemudian menghasilkan prediksi framework dan nilai probabilitas masing-masing kelas.



Hasil rekomendasi ditampilkan dalam bentuk framework utama dan alternatif. Informasi pendukung dapat ditampilkan agar pengguna memahami alasan sistem memberikan rekomendasi tersebut. Dengan demikian, sistem tidak hanya memberikan nama framework, tetapi membantu pengguna memahami hasil yang diperoleh.

3.14 Pengujian Sistem dan Validasi Pakar

Pengujian sistem dilakukan setelah sistem berbasis web selesai dikembangkan. Metode *black-box testing* akan digunakan karena pengujian difokuskan pada fungsi sistem berdasarkan masukan dan keluaran tanpa harus memeriksa kode program secara langsung.

Pengujian dilakukan terhadap fungsi utama sistem, mulai dari pengisian karakteristik proyek, pengiriman data, proses pengolahan data, pemanggilan model, sampai dengan penampilan hasil rekomendasi. Kemudian hasil rekomendasi dari sistem juga akan dibandingkan dengan hasil prediksi model secara langsung. Hal ini dilakukan untuk memastikan bahwa model yang telah dievaluasi dapat digunakan dengan benar ketika telah diintegrasikan ke dalam sistem.

Selain pengujian *black-box*, sistem juga akan divalidasi oleh pakar yaitu beberapa developer praktisi dan alumni Informatika. Uji validasi pakar (*expert judgment*) ini bertujuan untuk mengevaluasi kualitas rekomendasi

sistem berdasarkan kesesuaian *framework* dan rasionalitas penjelasan rekomendasi

3.15 Lingkungan Penelitian

Penelitian ini membutuhkan perangkat keras dan perangkat lunak untuk mendukung proses pengumpulan dataset, pengolahan data, pemodelan *machine learning*, pengembangan sistem, serta pengujian. Secara lengkap, perangkat penelitian akan ditunjukkan pada tabel berikut.

Tabel 3.4 Tabel Lingkungan Penelitian

Komponen	Lingkungan Penelitian
Device	Vivobook_AsusLaptop X409JP_A409JP
Processor	Intel(R) Core(TM) i5-1035G1 CPU @ 1.00GHz (1.19 GHz)
Graphic card	Intel(R) UHD Graphics (128 MB)
OS	Windows 11
Pengolahan Data	Python 3.13.0 (Pandas, Numpy)
Machine Learning	Google Colab CPU
Dataset	GitHub API
Code Editor	Visual Studio Code
Sistem berbasis Web	Python, HTML, CSS, JavaScript

DAFTAR PUSTAKA

[1] M. Absalon and T. Deneux, ""AlphAI": Teaching AI Algorithms to K12 by Training Learning Robots and Visualizing How It Works," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 28, pp. 29096–29103, Apr. 2025, doi: https://doi.org/10.1609/aaai.v39i28.35181.

[2] V. Cerqueira, L. Torgo, P. Branco, and C. Bellinger, "Automated imbalanced classification via layered learning," *Mach Learn*, vol. 112, no. 6, pp. 2083–2104, Jun. 2023, doi: 10.1007/s10994-022-06282-w.

[3] M. Carpita and S. Golia, "Categorical classifiers in multiclass classification with imbalanced datasets," *Statistical Analysis*, vol. 16, no. 4, pp. 391–405, Aug. 2023, doi: 10.1002/sam.11624.

[4] G. Robles, A. Capiluppi, J. M. Gonzalez-Barahona, B. Lundell, and J. Gamalielsson, "Development effort estimation in free/open source software from activity in version control systems," *Empir Software Eng*, vol. 27, no. 6, p. 135, Nov. 2022, doi: 10.1007/s10664-022-10166-x.

[5] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen, and R. Rubei, "Development of recommendation systems for software engineering: the CROSSMINER experience," *Empir Software Eng*, vol. 26, no. 4, p. 69, Jul. 2021, doi: 10.1007/s10664-021-09963-7.

[6] N. E. Gold and J. Krinke, "Ethics in the mining of software repositories," *Empir Software Eng*, vol. 27, no. 1, p. 17, Jan. 2022, doi: 10.1007/s10664-021-10057-7.

[7] M. Naiseh, D. Al-Thani, N. Jiang, and R. Ali, "Explainable recommendation: when design meets trust calibration," *World Wide Web*, vol. 24, no. 5, pp. 1857–1884, Sep. 2021, doi: 10.1007/s11280-021-00916-0.

[8] I. M. De Diego, A. R. Redondo, R. R. Fernández, J. Navarro, and J. M. Moguerza, "General Performance Score for classification problems," *Appl Intell*, vol. 52, no. 10, pp. 12049–12063, Aug. 2022, doi: 10.1007/s10489-021-03041-7.

[9] W. Pannakkong, K. Thiwa-Anont, K. Singthong, P. Parthanadee, and J. Buddhakulsomsiri, "Hyperparameter Tuning of Machine Learning Algorithms Using Response Surface Methodology: A Case Study of ANN, SVM, and DBN," *Mathematical Problems in Engineering*, vol. 2022, pp. 1–17, Jan. 2022, doi: 10.1155/2022/8513719.

[10] Z. Kotti, R. Galanopoulou, and D. Spinellis, "Machine Learning for Software Engineering: A Tertiary Study," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–39, Dec. 2023, doi: 10.1145/3572905.

[11] A. Pfob, S.-C. Lu, and C. Sidey-Gibbons, "Machine learning in medicine: a practical introduction to techniques for data pre-processing, hyperparameter tuning, and

model comparison," *BMC Med Res Methodol*, vol. 22, no. 1, p. 282, Nov. 2022, doi: 10.1186/s12874-022-01758-8.

[12] A. C. Darmawan and L. Iswari, "Pengembangan Aplikasi Berbasis Web dengan Python Flask untuk Klasifikasi Data Menggunakan Metode Decision Tree C4.5", *Journal Universitas Pahlawan,* vol. 4, no. 5, Oct. 2022, doi: https://doi.org/10.31004/jpdk.v4i5.7492

[13] S. Martínez-Fernández *et al.*, "Software Engineering for AI-Based Systems: A Survey," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 2, pp. 1–59, Apr. 2022, doi: 10.1145/3487043.

[14] Stack Overflow. (2025). *2025 Developer Survey: Web frameworks and technologies* [Set data]. Stack Overflow. https://survey.stackoverflow.co/2025/technology#1-web-frameworks-and-technologies

[15] Vidoni, "A systematic process for Mining Software Repositories: Results from a systematic literature review," *Inf. Softw. Technol.*, vol. 144, p. 106791, Apr. 2022, doi: https://doi.org/10.1016/j.infsof.2021.106791.

[16] L. Liao, H. Li, W. Shang, and L. Ma, "An Empirical Study of the Impact of Hyperparameter Tuning and Model Optimization on the Performance Properties of Deep Neural Networks," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 3, pp. 1–40, Jul. 2022, doi: 10.1145/3506695.

[17] S. zed, S. Jansen, and M. Deldar, "A decision model for programming language ecosystem selection: Seven industry case studies," *Information and Software Technology*, vol. 139, p. 106640, Nov. 2021, doi: 10.1016/j.infsof.2021.106640.

[18] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. Doi: https://doi.org/10.1023/A:1010933404324